RV UNIVERSITY

Go, change the world

an initiative of RV EDUCATIONAL INSTITUTIONS

NEW-AGE GLOBAL UNIVERSITY FOR LIBERAL EDUCATION

Semester: II

Category: Core

Operating System

Course Code: CS1103

(Theory and Lab)

Course Outline

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Unit – I	04 Hrs
Introduction to Operating Systems: Operating Systems –Definition Types- Functions -Abstract view of OS- System Structures – System Calls- Virtual Machines.	
Unit – II	08 Hrs
Process Management: Process Concepts, Inter-process communication –Threads –Multithreading, Process Scheduling, First-Come, First-Served (FCFS), Shortest Job First, Round Robin (RR), Priority Scheduling.	
Unit – III	06 Hrs
Synchronization and Deadlocks: Synchronization –Semaphores –Monitors, Hardware Synchronization – Deadlocks –Methods for Handling Deadlocks.	
Unit – IV	12 Hrs
Memory Management: Memory Management Strategies –Contiguous and Non-Contiguous allocation –Paging –Virtual memory Management –File System Design and Implementation – Mass Storage Structure –Disk Scheduling and Management, FCFS, SSTF, and SCAN -I/O Systems- Overview of System Protection and Security.	

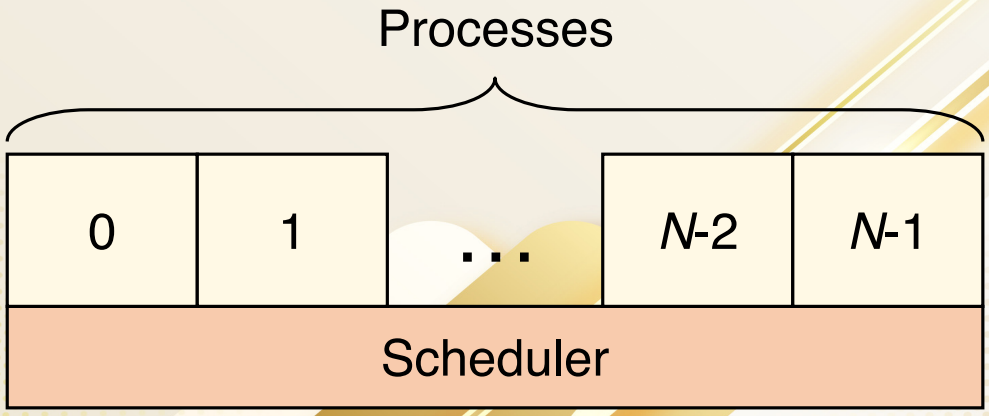
Contents

- Processes in the OS
- Process table entry
- Trap/ interrupt
- Threads: “processes” sharing memory
- Process & thread information
- Thread and stack
- Need for Threads
- Multithreaded Web server
- Interprocess communication
- Socket programming

Created by: Dr. Sangeeta Bhanja Chaudhuri

Processes in the OS

- Two “layers” for processes
  - Lowest layer of process-structured OS handles interrupts, scheduling
  - Above that layer are sequential processes
    - Processes tracked in the *process table*
    - Each process has a process table entry



Courtesy: CS 1550, cs.pitt.edu (originally modified by Ethan L. Miller and Scott A. Brandt)

What is in the process table entry?

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

May be stored on stack

Process management
- Registers
- Program counter
- CPU status word
- Stack pointer
- Process state
- Priority / scheduling parameters
- Process ID
- Parent process ID
- Signals
- Process start time
- Total CPU usage

File management
- Root directory
- Working (current) directory
- File descriptors
- User ID
- Group ID

Memory management
- Pointers to text, data, stack
  or
- Pointer to page table

Courtesy: CS 1550, cs.pitt.edu (originally modified by Ethan L. Miller and Scott A. Brandt)

Q. What do think, happens in a trap/ interrupt?

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

1. Hardware saves program counter (on stack or in a special register)
2. Hardware loads new PC, identifies interrupt
3. Assembly language routine saves registers
4. Assembly language routine sets up stack
5. Assembly language calls C to run service routine
6. Service routine calls scheduler
7. Scheduler selects a process to run next (might be the one interrupted…)
8. Assembly language routine loads PC & registers for the selected process

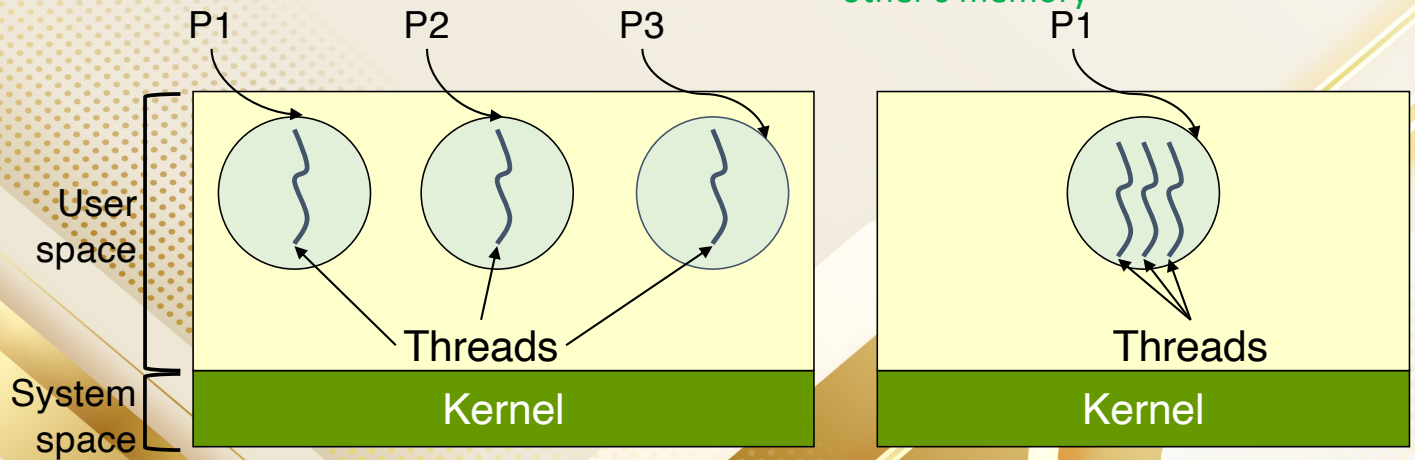
Courtesy: CS 1550, cs.pitt.edu (originally modified by Ethan L. Miller and Scott A. Brandt)

Threads: “processes” sharing memory

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Process (P) == address space
- Thread == program counter / stream of instructions
- Two examples are shown:
  - Three processes, each with one thread
  - One process with three threads

A thread within a process shares the same address space as the process itself, meaning all threads belonging to a single process can access the same memory locations without needing separate memory allocations for each thread; unlike processes which have their own distinct address spaces and cannot directly access each other's memory



Process & thread information

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

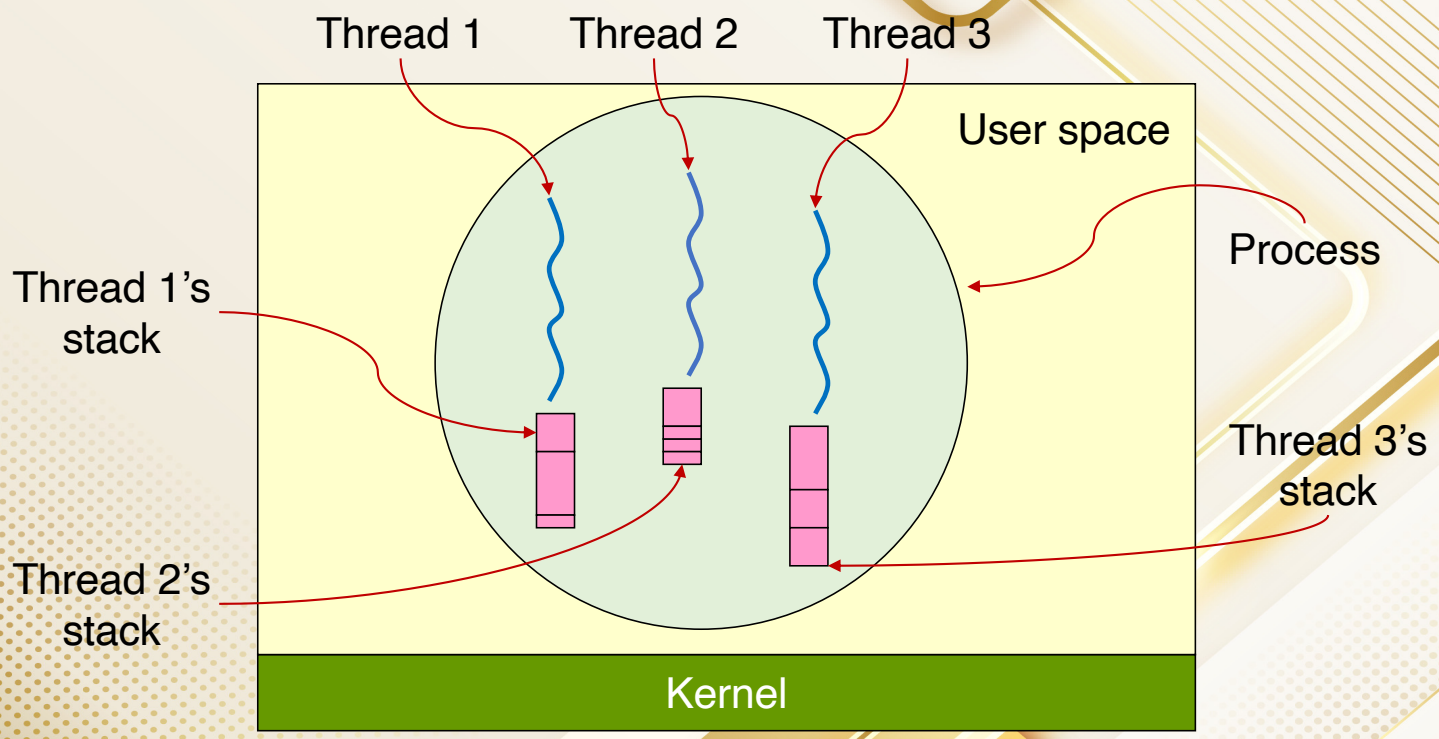
Per process items
Address space
Open files
Child processes
Signals & handlers
Accounting info
Global variables

Per thread items	**Per thread items**	**Per thread items**
Program counter	Program counter	Program counter
Registers	Registers	Registers
Stack & stack pointer	Stack & stack pointer	Stack & stack pointer
State	State	State

Courtesy: CS 1550, cs.pitt.edu (originally modified by Ethan L. Miller and Scott A. Brandt)

Threads & Stacks

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

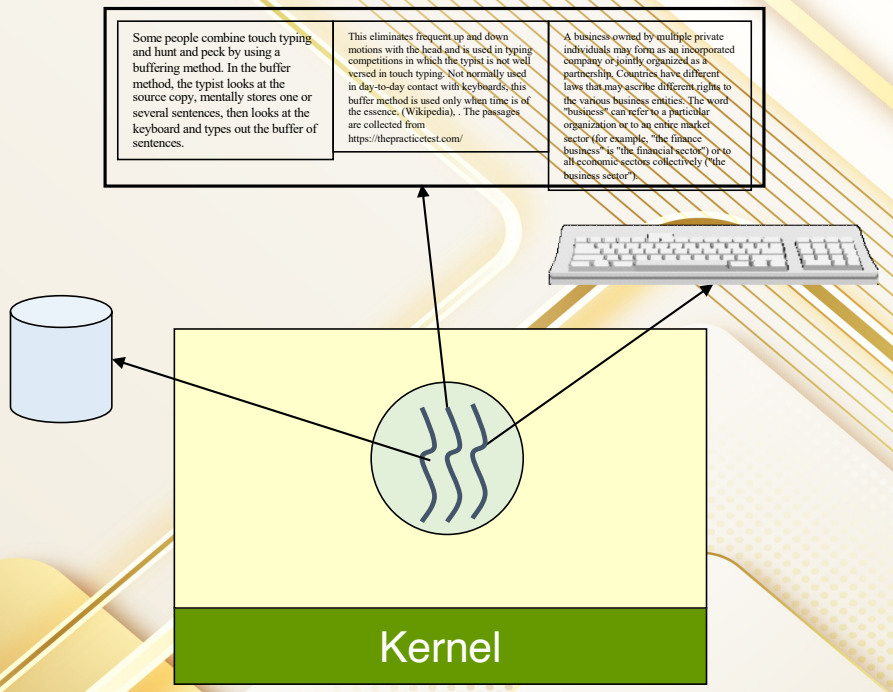


=> Each thread has its own stack!

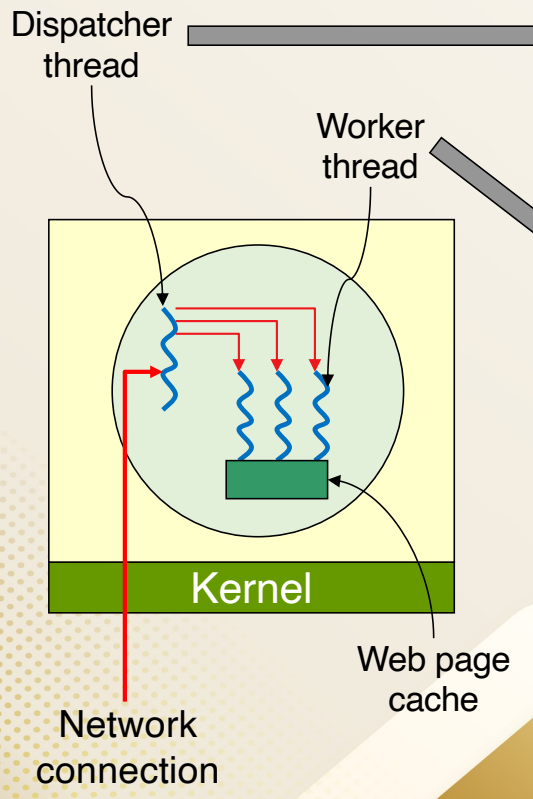
Q. What is the need for Threads?

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Achieve parallelism
- Allow a single application to do many things at once
  - Simpler programming model
  - Less waiting
- Threads are faster to create or destroy
  - No separate address space
- Overlap computation and I/O
  - Could be done without threads, but it's harder
- Example: word processor
  - Thread to read from keyboard
  - Thread to format document
  - Thread to write to disk



Multithreaded Web server




```
while(TRUE) {
  getNextRequest(&buf);
  handoffWork(&buf);
}
```

```
while(TRUE) {
  waitForWork(&buf);
  lookForPageInCache(&buf,&page);
  if(pageNotInCache(&page)) {
    readPageFromDisk(&buf,&page);
  }
  returnPage(&page);
}
```

Courtesy: CS 1550, cs.pitt.edu (originaly modified by Ethan L. Miller and Scott A. Brandt)

Explanation

- Infinite Loop (while(TRUE)):This loop runs indefinitely, constantly checking for new requests.
- getNextRequest(&buf);:This function is responsible for fetching the next available request. It could be reading from a queue, network socket, or some other source.
- handoffWork(&buf); :This function delegates the fetched request to another entity (likely a worker thread or a thread pool) for processing.

Explanation in terms of Threading Perspective:

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

This code is a typical structure for a producer-consumer model or a thread pool manager:

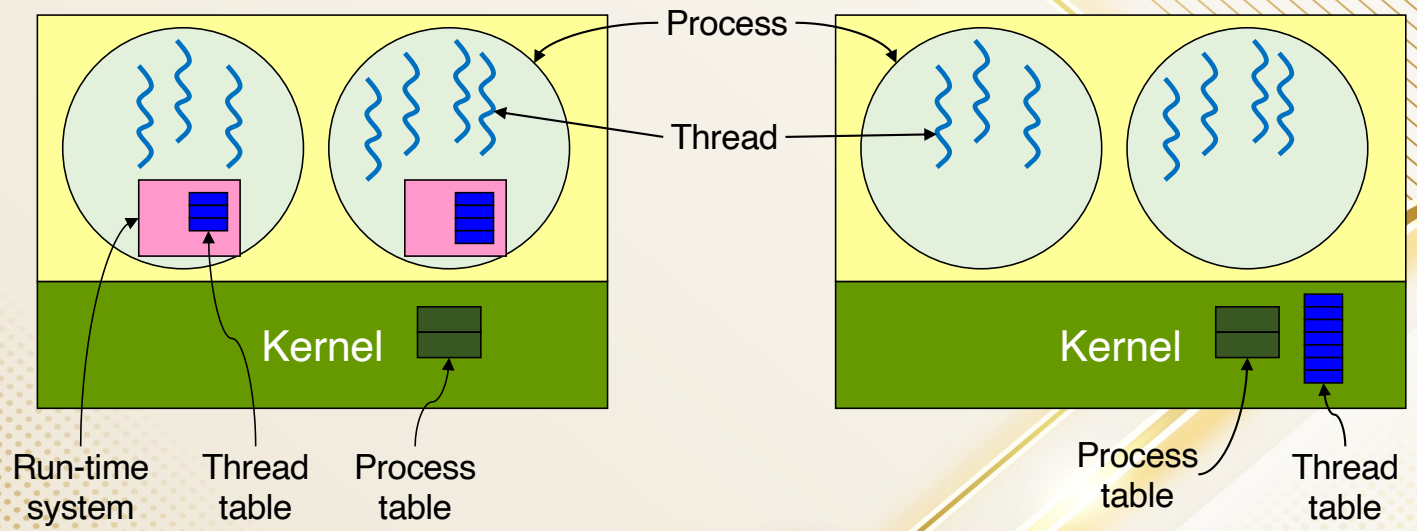
- Single-Threaded: If this loop runs in a single-threaded environment, all tasks will be handled sequentially, possibly causing delays.
- Multi-Threaded: If handoffWork() hands off the request to a worker thread (e.g., via a thread pool or task queue), this main thread acts as a dispatcher.
- Concurrency: This setup allows multiple worker threads to process requests in parallel, improving efficiency.

Three ways to build a server

- Thread model
  - Parallelism
  - Blocking system calls
- Single-threaded process: slow, but easier to do
  - No parallelism
  - Blocking system calls
- Finite-state machine
  - Each activity has its own state
  - States change when system calls complete or interrupts occur
  - Parallelism
  - Nonblocking system calls
  - Interrupts

Implement Thread

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS



User-level threads
+ No need for kernel support
- May be slower than kernel threads
- Harder to do non-blocking I/O

Kernel-level threads
+ More flexible scheduling
+ Non-blocking I/O
- Not portable

Interprocess Communication

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Processes within a system may be ***independent*** or ***cooperating***
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience
- Cooperating processes need **interprocess communication** (**IPC**)
- Two models of IPC
  - **Shared memory**
  - **Message passing**

Activity: Socket programming using TCP

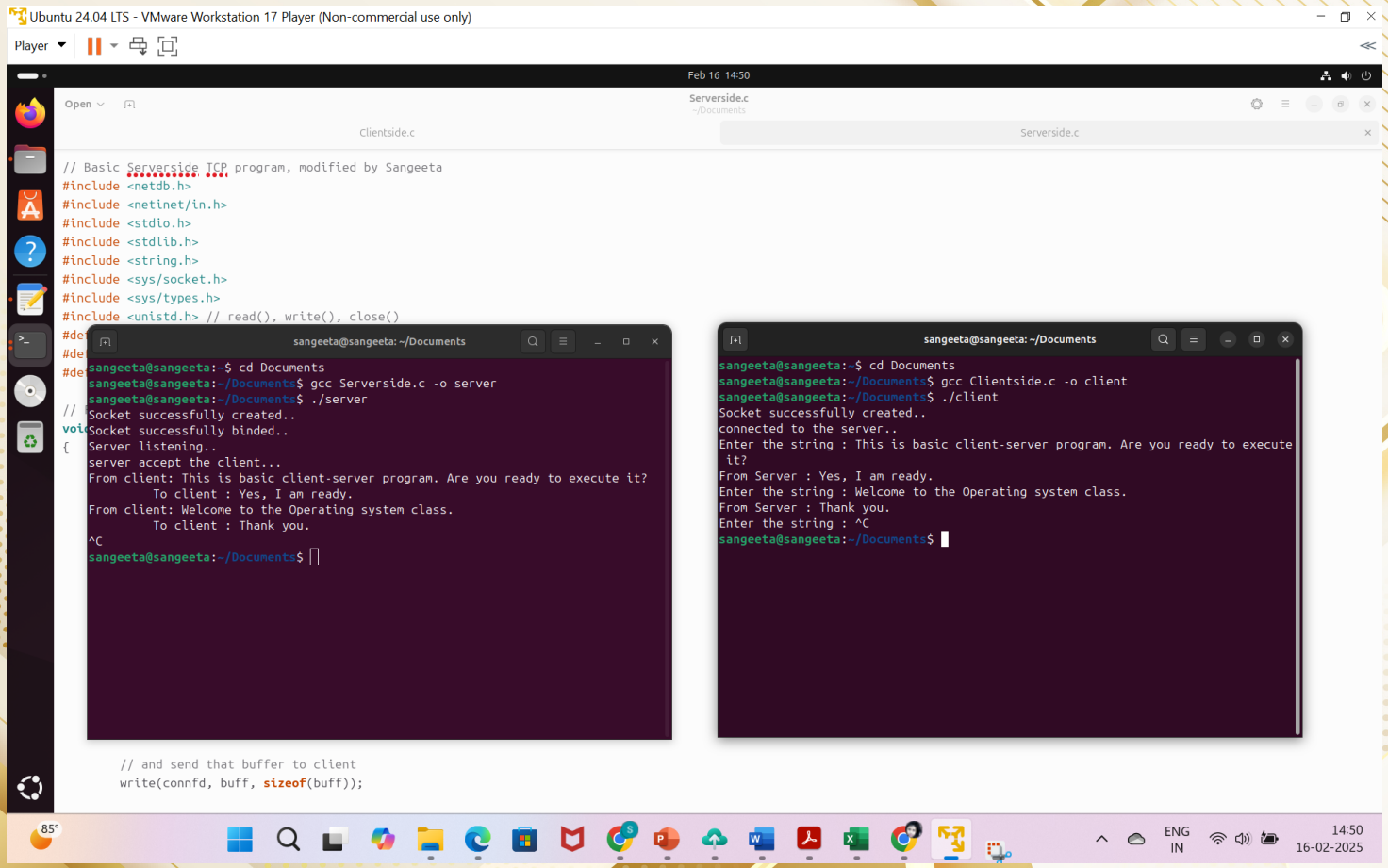
RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Code and documentation Link

Created by: Dr. Sangeeta Bhanja Chaudhuri

Activity: Socket Programming

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

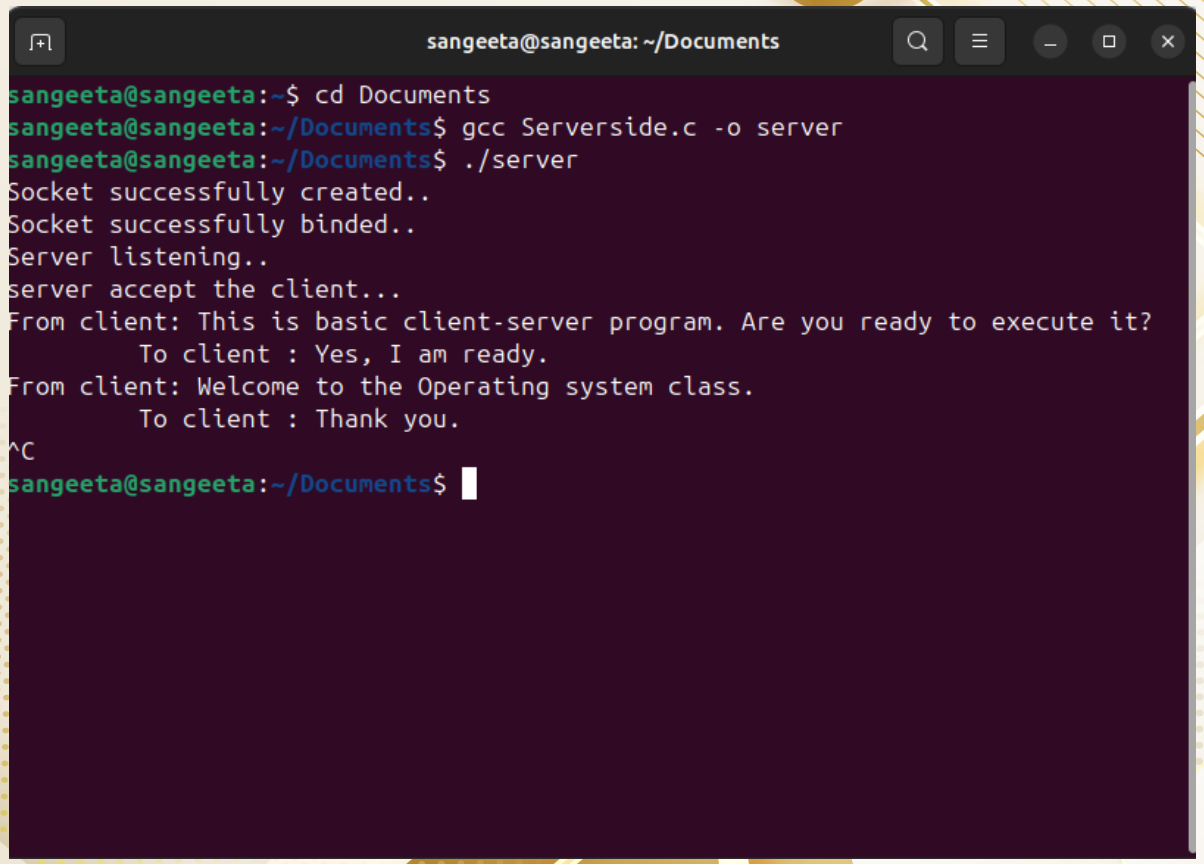


- TCP Server-Client implementation in C

Created by: Dr. Sangeeta Bhanja Chaudhuri

Server compilation and Execution

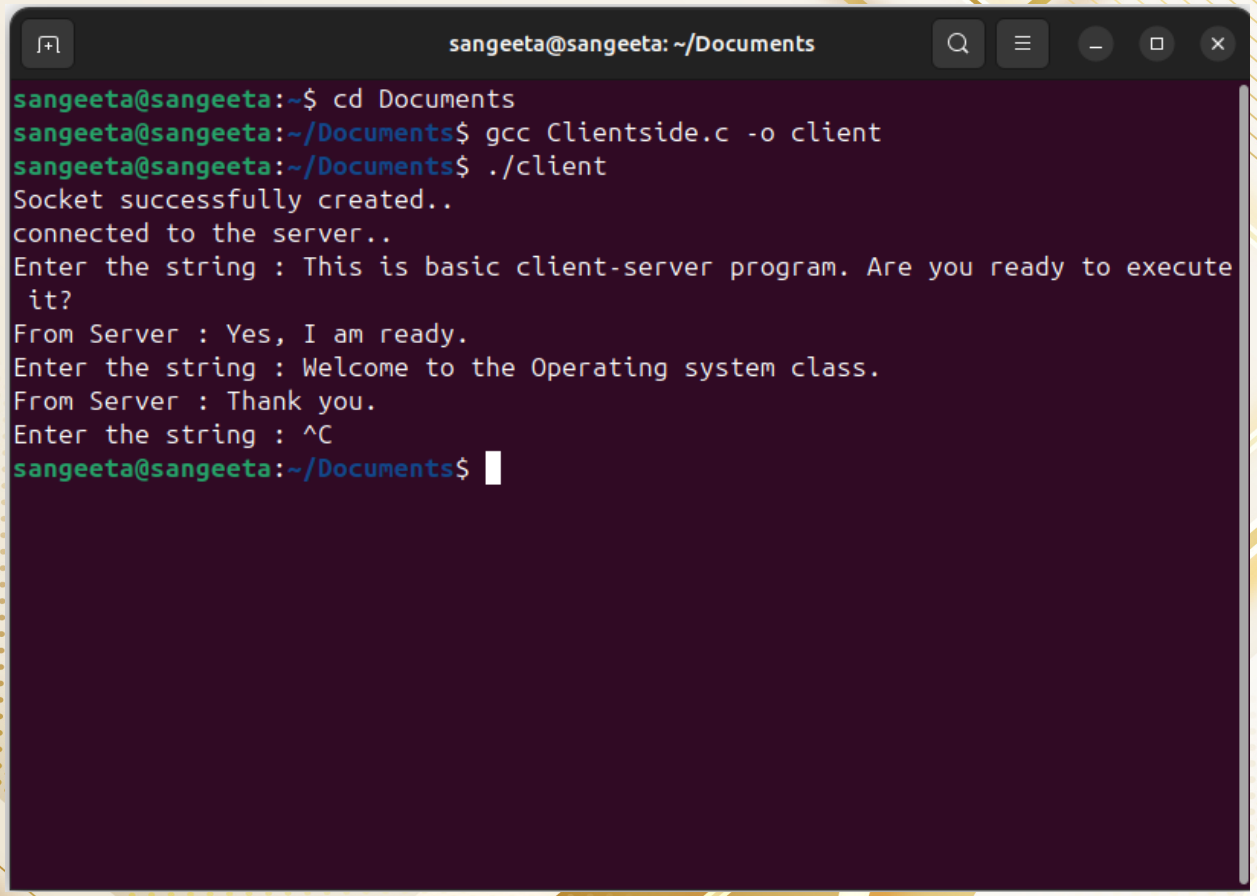
RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS



Created by: Dr. Sangeeta Bhanja Chaudhuri

Client compilation and Execution

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS



References

- Silberschatz, Galvin, Gagne, Operating System Concepts, John Wiley and Sons, 10th edition, 2023, ISBN: 935746056X
- Modern Operating Systems by Tanenbaum and Bos , 4th edition, Pearson Publisher, ISBN-13: 978-0-13-359162-0
- Courtesy: Prof. Mouli Sankaran, RV University.

Created by: Dr. Sangeeta Bhanja Chaudhuri